NAZARBAYEV UNIVERSITY

Numerical Methods with Applications
Math 351 – 1L,2L
Fall 2026

August 19, 2026

Roundoff errors



- Roundoff error is generally **inevitable** in numerical algorithms involving real numbers.



- People often like to pretend they work with **exact** real numbers, ignoring roundoff errors, which may allow concentration on other algorithmic aspects.



- However, **carelessness may lead to disaster!**



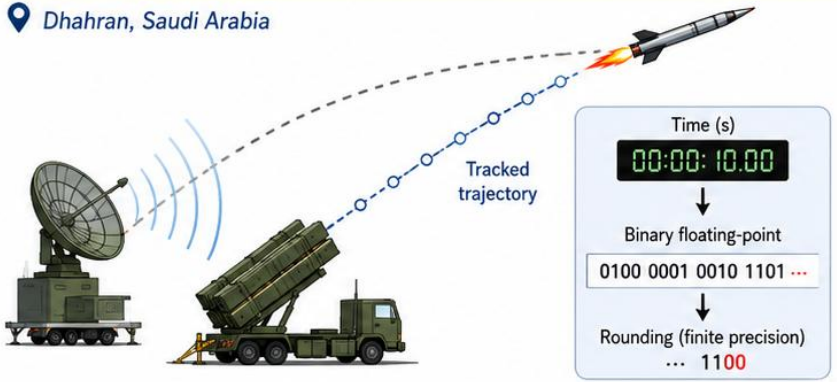
*Always be aware of roundoff errors and their possible impact on the **reliability** of numerical results.*

Some Disasters Attributable to Bad Numerical Computing

Real-world consequences of numerical and computational errors

1. Patriot Missile Failure (1991)

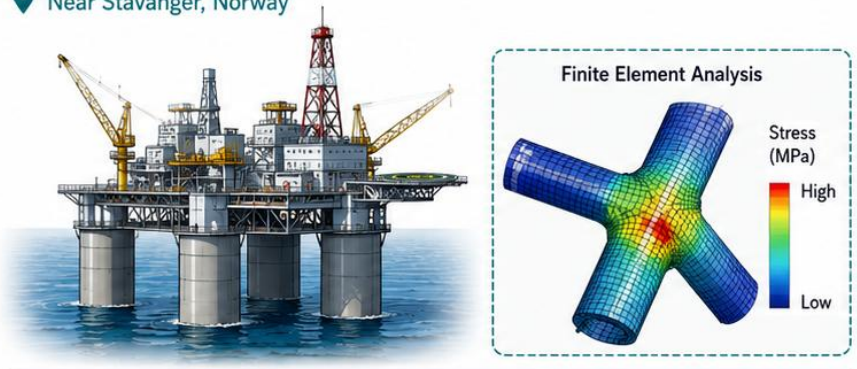
Dhahran, Saudi Arabia



- **Issue:** accumulated roundoff error in time calculation
- **Effect:** tracking error contributed to failure to intercept

2. Sleipner A Platform Failure (1991)

Near Stavanger, Norway



- **Issue:** inaccurate finite element analysis
- **Effect:** underestimated structural weakness led to failure

Real system Missile defense operation → **Numerical error** Roundoff in time computation → **Wrong prediction** Tracking drifts; missile misses → **Major consequence** 28 deaths

Real system Offshore platform → **Numerical error** Inaccurate FEA modeling → **Wrong prediction** Structural capacity underestimated → **Major consequence** ~$1 billion in losses

Key lesson: *Small numerical errors can lead to major real-world consequences.*

Adapted from numerical-disaster examples (University of Minnesota collection).

Source: https://www-users.cse.umn.edu/~arnold/disasters/disasters.ht

Real number representation: decimal

$$\frac{8}{3} \simeq \left(\frac{2}{10^0} + \frac{6}{10^1} + \frac{6}{10^2} + \frac{6}{10^3} \right) \times 10^0 = 2.666 \times 10^0.$$

An instance of the floating point representation

$$\begin{aligned} \mathrm{fl}(x) &= \pm d_0.d_1 \cdots d_{t-1} \times 10^e \\ &= \pm \left(\frac{d_0}{10^0} + \frac{d_1}{10^1} + \cdots + \frac{d_{t-2}}{10^{t-2}} + \frac{d_{t-1}}{10^{t-1}} \right) \times 10^e \end{aligned}$$

for $t = 4,\ e = 0$.
Note that $d_0 > 0$: normalized floating point representation.

Real number representation: binary

The decimal system is convenient for humans; but computers prefer binary.

- In binary the (normalized) representation of a real number x is

$$\begin{aligned} x &= \pm(1.d_1 d_2 d_3 \cdots d_{t-1} d_t d_{t+1} \cdots) \times 2^e \\ &= \pm(1 + \frac{d_1}{2} + \frac{d_2}{4} + \frac{d_3}{8} + \cdots) \times 2^e, \end{aligned}$$

  with binary digits $d_i = 0$ or 1 and exponent e.

- Floating point representation: with a fixed number of digits t

$$\mathrm{fl}(x) = \pm(1.\tilde{d}_1 \tilde{d}_2 \tilde{d}_3 \cdots \tilde{d}_{t-1} \tilde{d}_t) \times 2^e$$

- How to determine digits $\tilde{d}_i$?
  A popular strategy is **Rounding**:

$$\mathrm{fl}(x) = \begin{cases} \pm\, 1.d_1 d_2 d_3 \cdots d_t \times 2^e & d_{t+1} = 0 \\ \text{to nearest even} & \text{otherwise} \end{cases}.$$

  Alternatively, **Chopping** simply sets $\tilde{d}_i = d_i,\ i = 1, \ldots, t$.

General floating point system

defined by (β, t, L, U), where:
β: base of the number system (for binary, $\beta = 2$; for decimal, $\beta = 10$);
t: precision (number of digits);
L: lower bound on exponent e;
U: upper bound on exponent e.

For each $x \in \mathbb{R}$ corresponds a normalized floating point representation

$$\mathrm{fl}(x) = \pm \left(\frac{d_0}{\beta^0} + \frac{d_1}{\beta^1} + \cdots + \frac{d_{t-1}}{\beta^{t-1}} \right) \times \beta^e,$$

where $0 \leq d_i \leq \beta - 1$, $d_0 > 0$, and $L \leq e \leq U$.

Example

$(\beta, t, L, U) = (10, 4, -2, 1)$.
Largest number is $9.999 \times 10^U = 99.99 \lessapprox 10^{U+1} = 100$
Smallest positive number is $1.000 \times 10^L = 10^L = 0.01$
Total different fractions: $(\beta - 1) \times \beta^{t-1} = 9 \times 10 \times 10 \times 10 = 9,000$
Total different exponents: $U - L + 1 = 4$
Total different positive numbers: $4 \times 9,000 = 36,000$
Total different numbers in system: 72,001

Error in floating point number representation

For the real number $x = \pm\, d_0.d_1d_2d_3\cdots d_{t-1}d_td_{t+1}\cdots \times \beta^e$,

- Chopping:

$$\mathrm{fl}(x) = \pm\, d_0.d_1d_2d_3\cdots d_{t-1} \times \beta^e$$

Then absolute error is clearly bounded by $\beta^{1-t}\cdot\beta^e$.

- Rounding:

$$\mathrm{fl}(x) = \begin{cases} \pm\, d_0.d_1d_2d_3\cdots d_{t-1} \times \beta^e & d_t < \beta/2 \\ \pm\left(d_0.d_1d_2d_3\cdots d_{t-1} + \beta^{1-t}\right)\times\beta^e & d_t > \beta/2 \end{cases},$$

round to even in case of a tie.
Then absolute error is bounded by half the above, $\frac{1}{2}\cdot\beta^{1-t}\cdot\beta^e$.
So relative error is bounded by **rounding unit**

$$\eta = \frac{1}{2}\cdot\beta^{1-t}.$$

Floating point arithmetic

- Important to use **exact rounding**: if $\mathrm{fl}(x)$ and $\mathrm{fl}(y)$ are machine numbers, then

$$\begin{aligned} \mathrm{fl}(\mathrm{fl}(x) \pm \mathrm{fl}(y)) &= (\mathrm{fl}(x) \pm \mathrm{fl}(y))(1+\epsilon_1), \\ \mathrm{fl}(\mathrm{fl}(x) \times \mathrm{fl}(y)) &= (\mathrm{fl}(x) \times \mathrm{fl}(y))(1+\epsilon_2), \\ \mathrm{fl}(\mathrm{fl}(x)/\mathrm{fl}(y)) &= (\mathrm{fl}(x)/\mathrm{fl}(y))(1+\epsilon_3), \end{aligned}$$

  where $|\epsilon_i| \leq \eta$.

- Thus, the *relative errors* remain small after each such operation.

Example

$$a + (b + c) \neq (a + b) + c$$

Example

$$a + (b + c) \neq (a + b) + c$$

1. **Consider the sum of 3 numbers:** $y = a + b + c$.
2. **Done as** $fl(fl(a + b) + c)$

$$\begin{aligned}
\eta &= fl(a + b) &&= (a + b)(1 + \varepsilon_1) \\
y_1 &= fl(\eta + c) &&= (\eta + c)(1 + \varepsilon_2) \\
&= [(a + b)(1 + \varepsilon_1) + c](1 + \varepsilon_2) \\
&= [(a + b + c) + (a + b)\varepsilon_1](1 + \varepsilon_2) \\
&= (a + b + c)\left[1 + \frac{a + b}{a + b + c}\varepsilon_1(1 + \varepsilon_2) + \varepsilon_2\right]
\end{aligned}$$

3. **So disregarding the high order term** $\varepsilon_1\varepsilon_2$

$$fl(fl(a + b) + c) = (a + b + c)(1 + \varepsilon_3) \quad \textbf{with} \quad \varepsilon_3 \approx \frac{a + b}{a + b + c}\varepsilon_1 + \varepsilon_2$$

Example (continued)

1. If we redid the computation as $y_2 = fl(a + fl(b + c))$ we would find

$$fl(a + fl(b + c)) = (a + b + c)(1 + \epsilon_4) \quad \text{with} \quad \epsilon_4 \approx \frac{b + c}{a + b + c}\epsilon_1 + \epsilon_2$$

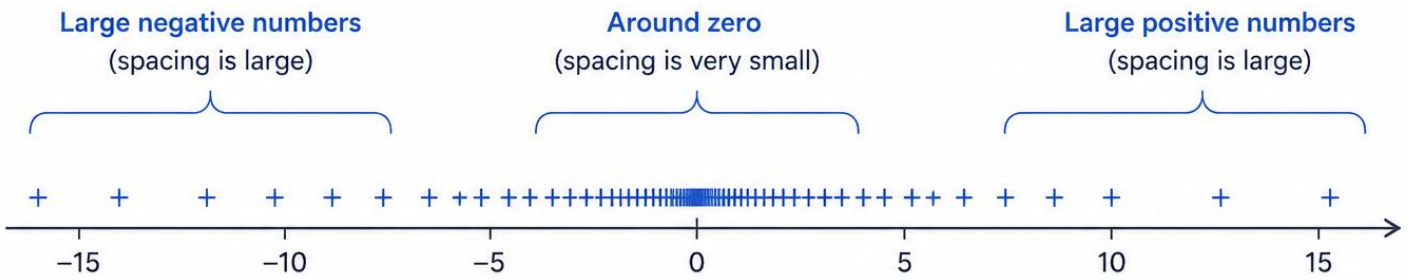
Main conclusion:

The first error is **amplified** by the factor $\frac{a+b}{y}$ in the first case and $\frac{b+c}{y}$ in the second case.

Takeaway: In order to sum n numbers more accurately, it is better to start with the small numbers first. However, sorting before adding is usually not worth the cost.

Spacing of floating point numbers

Floating point numbers are **not uniformly spaced** on the real line.
The distance between consecutive representable numbers **depends on the magnitude**.



Note the **uneven distribution**:
numbers are much more densely packed near 0 and more widely spaced as $|x|$ (the magnitude) increases.

Overflow, underflow, NaN

- Overflow: when $e > U$. (fatal)
- Underflow: when $e < L$. (non-fatal: set to 0 by default)
- NaN: Not-a-number. (e.g., $0/0$)

IEEE standard



Used by everyone today.



Binary: use $\beta = 2$.



Exact rounding: use guard digits to ensure that relative error in each elementary arithmetic operation is bounded by η.



NaN (Not-a-Number).



Overflow and underflow.



Subnormal numbers near 0.



Many other features...

IEEE Standard Word (Base $\beta = 2$)

Recall: In normalized numbers (base 2), the first digit is always $d_0 = 1$ and therefore it is not stored.

$$x = \underbrace{(-1)^s}_{\text{sign}} \times \underbrace{1.f_1 f_2 \dots f_t}_{\substack{\text{fraction} \\ \text{(mantissa)}}} \times \underbrace{2^e}_{\text{exponent}}$$

Double Precision (64-bit word)



$s = \pm$ $b = 11$-bit exponent $f = 52$-bit fraction

Rounding unit: $\eta = \frac{1}{2} \cdot 2^{-52} \approx 1.1 \times 10^{-16}$

All rounding errors are bounded by $|\text{error}| \leq \eta$.

Single Precision (32-bit word)

Can also use single precision (32-bit word):



$s = \pm$ $t = 8$-bit exponent $f = 23$-bit fraction

Rounding unit (single precision):

$$\eta = \frac{1}{2} \cdot 2^{-24} \approx 6.0 \times 10^{-8}$$

All rounding errors are bounded by $|\text{error}| \leq \eta$.

Key takeaway: Double precision provides much higher accuracy (rounding unit $\approx 10^{-16}$) compared to single precision (rounding unit $\approx 10^{-8}$).

IEEE Floating-Point Formats

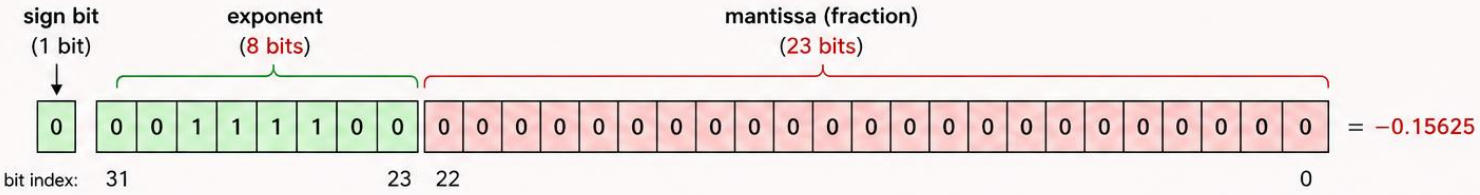
- 32-bit word lengths are typical.
- IEEE **Single-precision** floating-point numbers use **32 bits**.
- IEEE **Double-precision** floating-point numbers use **64 bits**.

Goal: Use the available bits to best represent the normalized floating-point number.

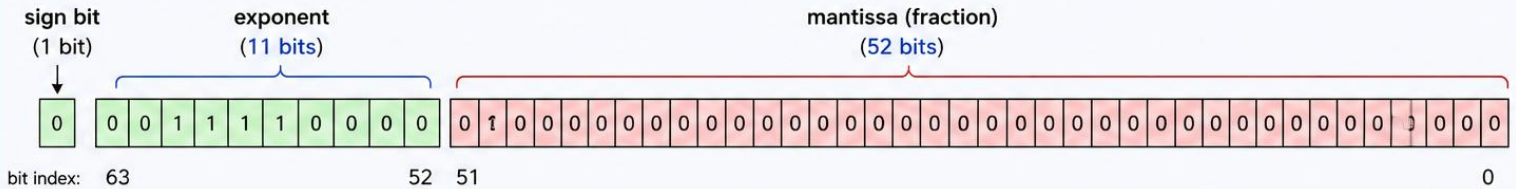
Normalized form (base 2)

$$x = \underbrace{(-1)^{s}}_{s \text{ sign}} \times \underbrace{1.f_1 f_2 \ldots f_t}_{\text{fraction (mantissa)}} \times \underbrace{2^{e}}_{e \text{ exponent (biased)}}$$

IEEE Single Precision (32-bit)

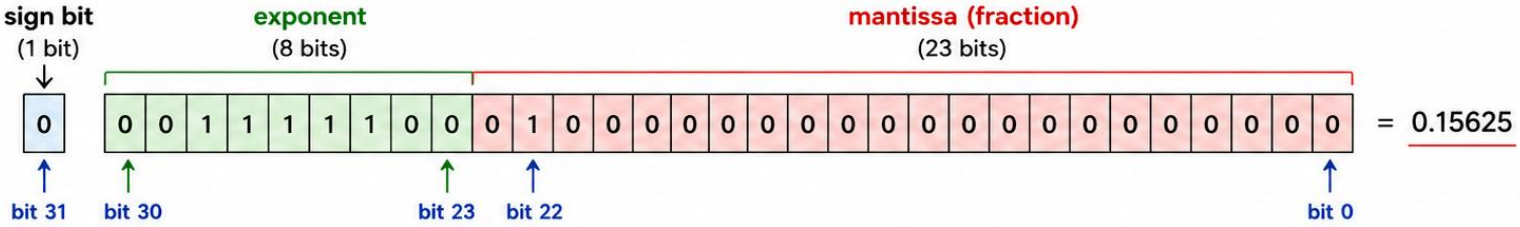


IEEE Double Precision (64-bit)



IEEE Single-Precision Representation (32-bit)

$$x = \pm q \times 2^m$$



Notes

1. 1-bit sign: 0 for positive numbers, 1 for negative numbers.
2. 8-bit exponent encodes m (the power of 2).
3. 23-bit mantissa q (the fraction).
4. The leading one in the mantissa q is not stored: $b_1 = 1$ is the **hidden** (implicit) bit.
5. IEEE 754: numbers are stored in normalized form $1, f$, i.e., $1 \leq q < 2$ (with the hidden leading one).
6. Exponent is stored with bias (offset) of 127:

   $$c = m + 127 \quad \text{(stored value)}$$

7. Special cases: $c = 0$ and $c = 255$ (see right panel).

Exponent (biased)

$$c = m + 127, \quad 0 \leq c \leq 255$$

Stored value c	Actual exponent m
1	$m = -126$ (smallest)
254	$m = +127$ (largest)

Ranges

- Largest exponent: $m = +127$
- Smallest exponent: $m = -126$

Special cases

- $c = 0$ → denormals (subnormals) or zero
- $c = 255$ → infinity or NaN (not a number)

IEEE Floating-Point Conversion Example

Process for $x = -52.125$:

$$x = \pm q \times 2^{m}$$

1 Convert both integer and fractional to binary:

$$x = -\underbrace{(110100.}_{\text{integer part}}\underbrace{00100000000)_2}_{\text{fractional part}}$$

2 Convert to $1.f$ form:

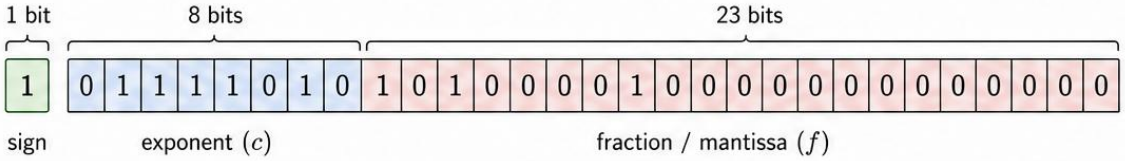
$$x = -(\underbrace{1}_{\text{1 (hidden)}}.\ \underbrace{1010000100000)_2}_{\text{23 fraction bits}} \times 2^5$$

3 Convert exponent 5:

$$c = 127 - 5 = 122 \Rightarrow c = \underbrace{(01111010)_2}_{\text{8 bits}}$$

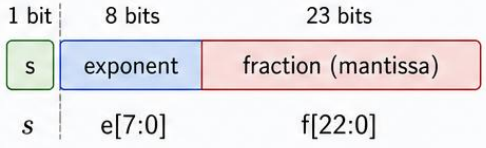
$$\underbrace{1}_{\text{1 (hidden)}}\ \underbrace{10100001}_{\text{8 bits}}\ \underbrace{00000}_{\text{23 bits}}$$

4 Final representation (32-bit IEEE single precision):



$$= (0.1010000100000)_2 \times 2^5$$

IEEE single-precision (32-bit)



Legend:

- s – sign bit (0 = positive, 1 = negative)
- e – exponent (bias = 127)
- f – fraction (mantissa)

Special Cases in IEEE Floating-Point (32-bit / Single Precision)

When the exponent field is all 0s or all 1s, special encodings are used.

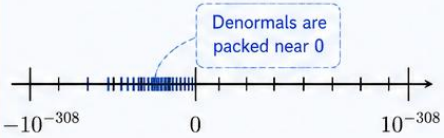
Case	Sign (1 bit)	Exponent (8 bits)	Mantissa / Fraction (23 bits)	Value / Meaning
Denormalized (Subnormal) Numbers	s	00000000	$\neq 000\ldots000$ (nonzero)	$\pm 0.\text{fraction} \times 2^{-126}$ (denormalized numbers) • Uses 1 extra bit in significand • Exponent is fixed at -126 • Less precision, more range
$+0$	0	00000000	$000\ldots000$	$+0$ (exact zero)
-0	1	00000000	$000\ldots000$	-0 (exact zero)
$+\infty$	0	11111111	$000\ldots000$	$+\infty$ (positive infinity)
$-\infty$	1	11111111	$000\ldots000$	$-\infty$ (negative infinity)
NaN (Not a Number)	s	11111111	$\neq 000\ldots000$ (any nonzero)	NaN (not a number) Used to indicate undefined or unrepresentable results.

Denormalized / Subnormal Numbers

When exponent $= 00000000$ (biased exponent $= -126$):

- No implicit leading 1.
- The significand has the form $0.\text{fraction}$ (instead of $1.\text{fraction}$).
- Value represented as:

$$(-1)^s \times 0.\text{fraction} \times 2^{-126}$$



Denormals provide **gradual underflow** to zero, improving numerical stability at the cost of precision.

Reference: For full details see Appendix C.1 in *Numerical Methods for Engineers, 6th ed.*

Floating-Point Ranges (IEEE 754)

Type	Range of Representable Numbers	Approximate Range (Base 2)
single (32-bit) 1-8-23	$-3.40 \times 10^{38} \leq x \leq -1.18 \times 10^{-38}$ (negative normal range) 0 (zero) $1.18 \times 10^{-38} \leq x \leq 3.40 \times 10^{38}$ (positive normal range)	$2^{-126} \rightarrow 2^{128}$
double (64-bit) 1-11-52	$-1.80 \times 10^{308} \leq x \leq -2.23 \times 10^{-308}$ (negative normal range) 0 (zero) $2.23 \times 10^{-308} \leq x \leq 1.80 \times 10^{308}$ (positive normal range)	$2^{-1022} \rightarrow 2^{1024}$

S **Sign bit (1 bit):**
0 for positive,
1 for negative

E **Exponent**
(8 bits for single,
11 bits for double)

M **Mantissa / Fraction**
(23 bits for single,
52 bits for double)

Note:
Approximate range is shown in powers of 2 (base 2).

Listing 1: MATLAB Tools

```
>> realmax          % largest positive finite number
ans =  1.7977e+308
```

This is approximately $(1 - 2^{-53}) \times 2^{1024}$

```
>> realmin          % smallest positive normalized number
ans =  2.2251e-308
```

This is approximately 2^{-1022}

Key takeaway: Double precision provides a much wider range and higher precision than single precision.

Machine Epsilon

The smallest floating-point number that changes 1 when added

Definition

The machine epsilon ε_m is the smallest number such that

$$fl(1 + \varepsilon_m) \neq 1$$

- Double precision machine epsilon is approximately $2^{-52} \approx 2.2204 \times 10^{-16}$
- Single precision machine epsilon is approximately $2^{-23,} \approx 1.1921 \times 10^{-7}$

MATLAB Example

```
>> eps
ans = 2.2204e-16

>> eps('single')
ans = 1.1921e-07
```

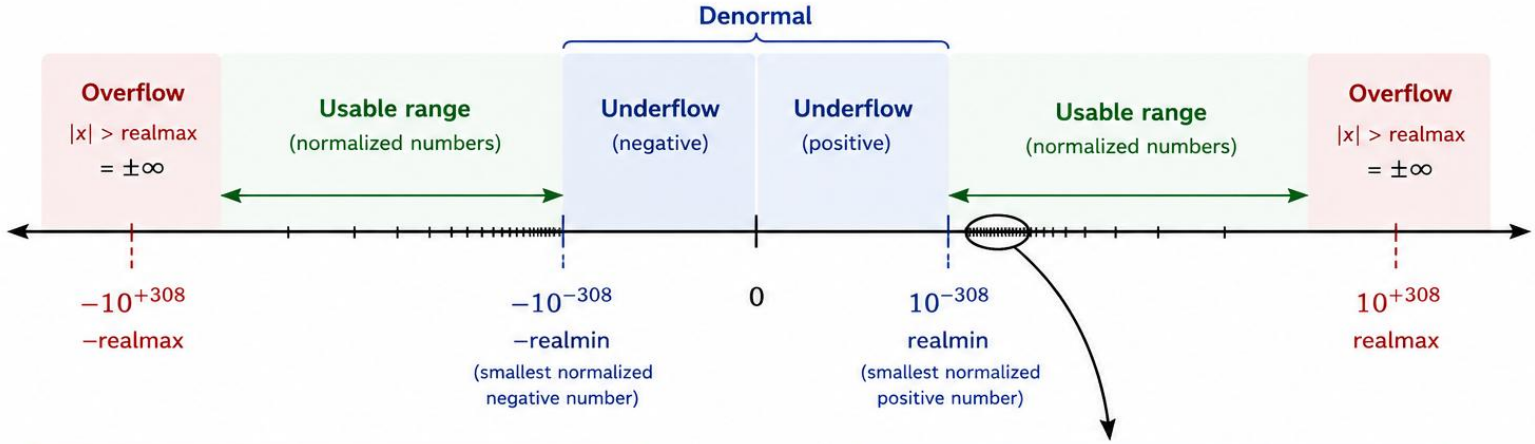
Comparison Near 1

Double Precision (64-bit)	Single Precision (32-bit)
$1 + 10^{-16}$ > 1 (distinguishable)	$= 1$ (not distinguishable)
$1 + 10^{-8}$ > 1 (distinguishable)	$= 1$ (not distinguishable)
$1 + 10^{-7}$ > 1 (distinguishable)	> 1 (distinguishable)

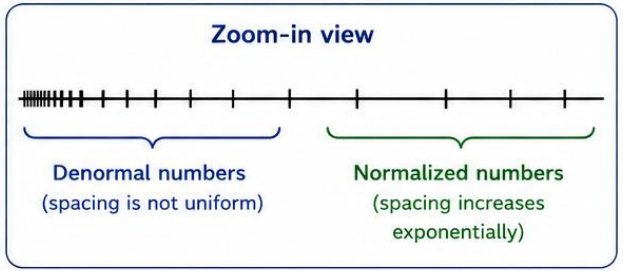


Takeaway: Machine epsilon measures the spacing of floating-point numbers near 1 and indicates the limit of relative precision.

Floating Point Number Line (IEEE 754, Double Precision)



- **Normalized numbers (usable range):** Represent the vast majority of real numbers.
- **Denormal numbers (underflow region):** Very small magnitude numbers close to zero. Provide gradual underflow to zero.
- **Overflow region:** Values too large in magnitude → ±∞ (infinity).



Key idea: Floating-point numbers are not evenly spaced.
They are dense near 0 (denormal region) and spread out as |x| increases (exponentially).

Roundoff error accumulation

1. In general, if E_n is the error after n elementary operations, then linear roundoff error accumulation is unavoidable.

   Acceptable (linear growth) $\quad E_n \simeq c_0 n E_0$

   E_n : error after n operations
   E_0 : initial error
   c_0 : constant independent of n

2. Will not tolerate **exponential** error growth such as

   Unacceptable (exponential growth) $\quad E_n \simeq c_1^n E_0$

   for some constant $c_1 > 1$

   ➜ This indicates an **unstable algorithm**.

3. In some situations, an individual error contribution can be particularly large and can sometimes be reduced.

Example: Roundoff Error in Single Precision



We study the function

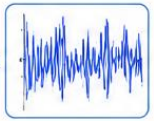
$$g(t) = e^{-t}(\sin(2\pi t) + 2)$$

on the interval $0 \le t \le 1$.

- Sample at 501 equally spaced points:
  $t = 0 : 0.002 : 1$
- Evaluate using `double` precision
- Evaluate using `single` precision

MATLAB Implementation

```
t  = 0:.002:1;                    % 501 points
tt = exp(-t).*(sin(2*pi*t)+2);    % double precision (reference)
rt = single(tt);                  % single precision
round_err = (tt - rt)./tt;        % relative roundoff error
plot(t, round_err, 'b-');
title('error in sampling exp(-t)(sin(2\pi t)+2) in single precision')
xlabel('t'), ylabel('roundoff error')
```

Exact function $g(t)$ → Double precision (tt) (reference) → Single precision (rt) → Relative roundoff error (tt-rt)/tt → 

Goal

Compare more accurate (double precision) values with less accurate (single precision) values to reveal roundoff error.

Relative Roundoff Error

$$\text{round_err} = \frac{tt - rt}{tt}$$

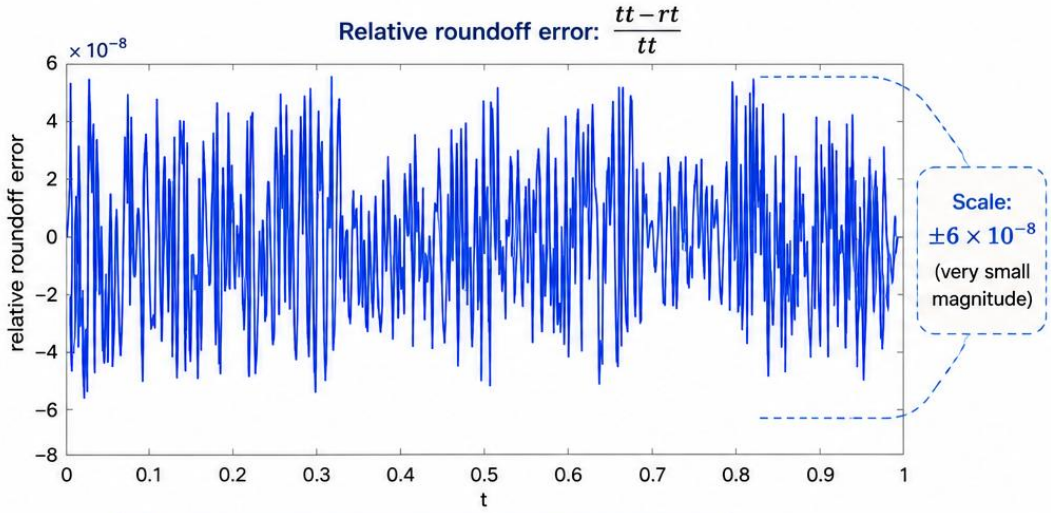
`tt` : values computed in double precision (reference)
`rt` : values computed in single precision
`round_err` : relative roundoff error

Takeaway

Because single precision stores fewer significant digits, the computed values differ slightly from double precision values. This difference is the **roundoff error**.

Observed Roundoff Error: Irregular and Non-Smooth

Relative roundoff error for $g(t) = e^{-t}(\sin(2\pi t) + 2)$ computed in single precision



The error changes sign rapidly and appears irregular. It behaves like high-frequency noise.

Key Observations

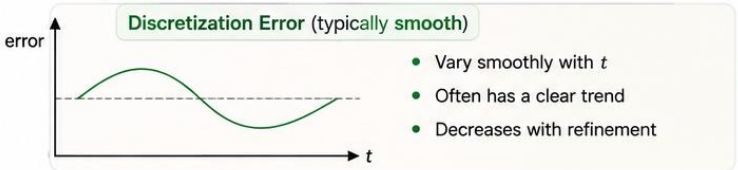
- The error magnitude is very small, on the order of 10^{-8}.
- The sign changes rapidly from point to point.
- The error is irregular and non-smooth, with no visible pattern.
- This is a typical characteristic of roundoff error due to finite precision arithmetic.

Machine Epsilon (single precision)

$$\varepsilon_{\text{single}} \approx 1.19 \times 10^{-7}$$

Observed relative error is of the same order.

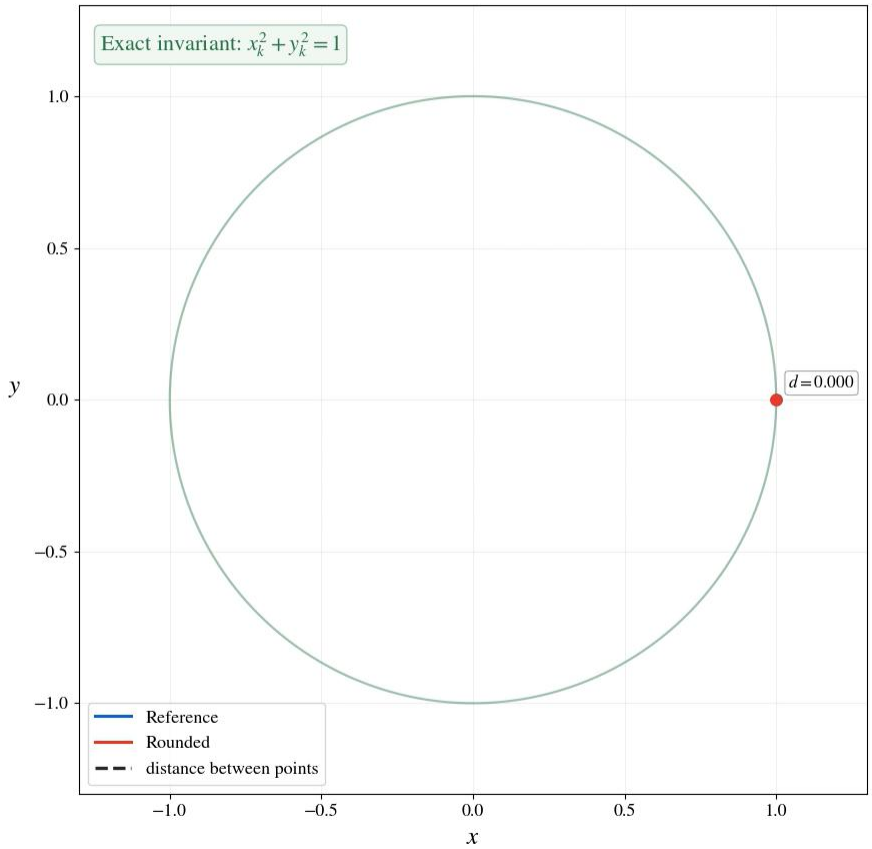
Roundoff Error vs. Discretization Error



Takeaway: Roundoff error behaves like small, high-frequency noise, while discretization error is usually smooth.

Roundoff Error in Repeated Rotation — Clarified View

Blue = reference trajectory, Red = rounded trajectory, rounding to 2 decimal places after every step

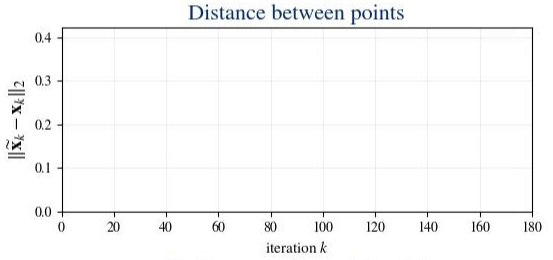


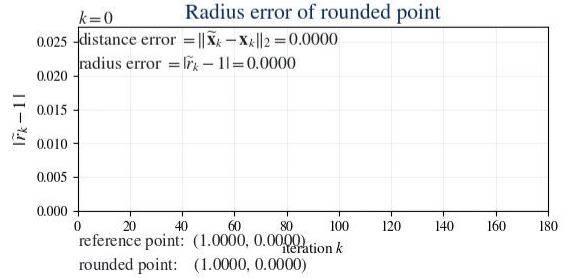
Repeated rotation

$$x_{k+1} = \cos\theta\, x_k - \sin\theta\, y_k$$
$$y_{k+1} = \sin\theta\, x_k + \cos\theta\, y_k$$

1. Nearly identical at first

Both trajectories start from the same point.
The distance error is still very small.





NAZARBAYEV
UNIVERSITY

